SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL – ASSIGNMENT 2

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO2	**Assessment:**	Assignment 2 –Informed, Adversarial and Constraint Search
Weightage:	_2% of CIA	**Duration / Marks:**	50 marks
CO2 Statement:	Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		
Student Name:	Tejhash P	**Reg. No.:**	192512495
Section / Batch:	1	**Date:**	11/10/2026

Code of Conduct

I Tejhash P, 192512495 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Tejhash.P

Instructions

- Read each task carefully before starting. This is a take-home assignment completed over one week.
- Each task is a problem drawn from the prescribed textbooks (Russell and Norvig, Alpaydin, Manning et al.). Select the technique that fits, apply it step by step, and support every decision with evidence.
- Tasks are drawn from Informed Search (Greedy, A*, heuristics), Local Search and Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta, evaluation functions).
- Every implemented task must be supported by: Source Code → Test Input → Expected Behaviour → Actual Output → Performance Metrics → Interpretation. Where a task says Predict, write the prediction before running the experiment; use fixed random seeds. Programming language: Python 3 (Jupyter / Colab).
- Show intermediate tables (expansion orders, domains, α/β values, operation counts) and state the final answer clearly.
- Marks will be awarded using the Assignment rubric: conceptual understanding, application of concepts, analytical thinking, organization & presentation, and completeness & accuracy.
- Upload your work in GCR and Github.

Reference Books Note

- T1 – Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach, 3rd Edition, PHI / Pearson Education (AIMA).
- Chapters used: Section 3.5–3.6 (greedy best-first search and A*, the straight-line distances of Figure 3.22, Figures 3.23 and 3.24); Section 4.1 (hill climbing, simulated annealing and the 8-queens problem); Section 4.5 (online search agents, Figures 4.21 and 4.24); Chapter 5 (Section 5.2–5.3 minimax and alpha-beta pruning, Figures 5.2 and 5.5, Exercise 5.9 on tic-tac-toe); Chapter 6 (Section 6.1–6.5 constraint satisfaction, map colouring of Australia in Figure 6.1, arc consistency, min-conflicts, Exercise 6.11).
- Figure, table, section and exercise numbers in this assignment refer to the editions used for the problems: AIMA 3rd edition (T1). The Romania distances and the Figure 3.22 heuristic values are reproduced from the book.

Section / Topic	Focus	Task Nos.

1. Informed and Online Search	Greedy best-first search, A*, heuristic functions (admissibility, consistency), online search agents	**Tasks 1–2**
2. Adversarial Search	Minimax, alpha-beta pruning and move ordering, evaluation functions (tic-tac-toe)	**Tasks 3–4**
3. Constraint Satisfaction and Local Search	CSP (arc consistency, forward checking, MRV, degree), hill climbing, simulated annealing, 8-queens	**Tasks 5–6**
4. Day-4 Design Challenge	Diagnose, redesign and validate a solver for efficiency (n-queens)	**Task 7**

Rubrics for Assignment

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	**25%**	Demonstrates complete and accurate understanding of concepts with strong clarity	Shows good understanding with minor gaps	Partial understanding with some errors	Lacks basic understanding of concepts
Application of Concepts	**25%**	Applies concepts correctly to problems with logical and appropriate solutions	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts correctly
Analytical Thinking	**20%**	Provides in-depth analysis with strong reasoning and justification	Shows logical analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Organization & Presentation	**15%**	Well-structured, neat, and logically organized with clear flow	Generally organized with minor issues	Some structure but lacks clarity	Poorly organized and difficult to follow
Completeness & Accuracy	**15%**	All questions attempted with accurate and error-free answers	Most answers correct with minor mistakes	Some answers incomplete or incorrect	Many answers missing or incorrect

Task / Criteria	Conceptual Understanding (25%)	Application of Concepts (25%)	Analytical Thinking (20%)	Organization & Presentation (15%)	Completeness & Accuracy (15%)	Sub. Total (Max)	Total %
Task 1 (6)						__ / 6	
Task 2 (4)						__ / 4	
Task 3 (6)						__ / 6	
Task 4 (6)						__ / 6	
Task 5 (7)						__ / 7	
Task 6 (8)						__ / 8	
Task 7 (13)						__ / 13	
Total (50)	__ / 12.5	__ / 12.5	__ / 10	__ / 7.5	__ / 7.5	__ / 50	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____

Date: _____	Date: _____	Date: _____

Task 1: Experiment with Greedy Best-First Search and A* Using Heuristic Functions under Different Map Conditions (6 Marks)

CO2 Topic Mapping: Topic 1 – Informed Search Strategies; Topic 2 – Greedy Best-First Search Algorithm; Topic 3 – A Search Algorithm; Topic 4 – Heuristic Functions and Heuristic Search*

Problem Context – Romania Route Finding (AIMA +3.5)

The agent of Assignment 1 wants the shortest route from Arad to Bucharest. In addition to the road distances of Figure 3.2, it can use the straight-line distance to Bucharest, hSLD (Figure 3.22):

City	h	City	h	City	h	City	h
Arad	366	Fagaras	176	Mehadia	241	Sibiu	253
Bucharest	0	Giurgiu	77	Neamt	234	Timisoara	329
Craiova	160	Hirsova	151	Oradea	380	Urziceni	80
Drobeta	242	Iasi	226	Pitesti	100	Vaslui	199
Eforie	161	Lugoj	244	Rimnicu Vilcea	193	Zerind	374

The development team is considering greedy best-first search ($f = h$), A* ($f = g + h$) and a weighted variant ($f = g + W \cdot h$).

Your Challenge

Experiment with the three algorithms, using heuristic functions, to determine whether one of them should always be preferred or whether the appropriate choice depends on the characteristics of the map, so that routes are found efficiently.

(a) Predict, before running anything, the order of expansion, the route and its cost for greedy search and for A* from Arad to Bucharest; then verify against AIMA Figures 3.23 and 3.24 (greedy: 450 km; A*: 418 km)

(b) Prove that hSLD is admissible and check the consistency inequality $h(n) \leq c(n, n') + h(n')$ on all 23 roads in both directions

(c) Design three deliberately different map conditions (for example, remove a road, change road costs so that the straight-line distance is no longer a good guide, add a dead end) and show where each algorithm succeeds or fails

(d) Run weighted A* with $W \in \{1, 1.2, 1.5, 2, 3\}$: record route cost, nodes expanded and the smallest W at which the route stops being optimal

(e) Compare A* with uniform-cost search on the same pairs and report the nodes saved by the heuristic

Evidence Expected: Reasoned prediction, a proof of admissibility and consistency, three designed maps, implementation, expansion orders, weighted-A* table and interpretation.

Task 2: Experiment with Online Search Agents: When Is Online Search an Appropriate Engineering Choice? (4 Marks)

CO2 Topic Mapping: Topic 6 – Online Search Agents

Problem Context – Exploring an Unknown Map (AIMA Section 4.5)

Suppose the agent does not know the Romania map in advance: at each city it learns only the roads leaving it, and it must act while exploring. Only an online algorithm can be used (the online DFS agent of AIMA Figure 4.21 and LRTA* of Figure 4.24).

(a) Design experiments in which the agent starts from each of the 20 cities with Bucharest as the goal; record the distance travelled and the competitive ratio against the optimal route found with a known map

(b) Repeat with two one-way roads (irreversible actions) added to the map and analyze when the agent reaches a dead end

(c) Compare the online DFS agent and LRTA* over repeated trials and record how the learned heuristic values change

(d) Evaluate when online search is the appropriate engineering choice rather than offline planning

Evidence Expected: Designed experiments, distance and competitive-ratio evidence, analysis of successful and failed runs and a recommendation supported by the data.

Task 3: Experiment with Alpha-Beta Pruning: Engineer Its Behaviour on Game Trees (6 Marks)

CO2 Topic Mapping: Topic 9 – Alpha-Beta Pruning and Evaluation Functions

Problem Context – The Two-Ply Game Tree of AIMA Figure 5.2

A root MAX node has three MIN children with leaf utilities (3, 12, 8), (2, 4, 6) and (14, 5, 2). Minimax gives the values 3, 2 and 2 for the children and 3 for the root. Alpha-beta pruning evaluates fewer leaves (AIMA Figure 5.5).

Instead of merely implementing the algorithm, experiment with alpha-beta pruning to determine how the structure of the game tree (move ordering) influences computational effort, so that adversarial search is carried out efficiently.

(a) Implement minimax and alpha-beta pruning; verify the Figure 5.2 values and count the leaves each algorithm evaluates on this tree

(b) Design three trees (best-case ordering, worst-case ordering and random ordering) with branching factor $b \in \{2, 3, 4\}$ and depth $d \in \{4, 6, 8\}$; predict the number of evaluated leaves before executing

(c) Compare the measured counts with b^d, with the best-case bound O(b^(d/2)) and with the random-ordering estimate O(b^(3d/4)) given in AIMA Section 5.3

(d) Record the cut-offs (α-cuts and β-cuts) and determine at which depths most pruning happens

Evidence Expected: Correct implementation, three designed trees, predictions before execution, node and cut-off counts and a comparison with the theoretical bounds.

Task 4: Experiment with Minimax and Evaluation Functions: The Cost of Examining Every Possibility (6 Marks)

CO2 Topic Mapping: Topic 8 – Adversarial Search and Minimax Algorithm; Topic 9 – Alpha-Beta Pruning and Evaluation Functions

Problem Context – Tic-Tac-Toe (AIMA Section 5.2 and Exercise 5.9)

The tic-tac-toe game tree has fewer than 9! = 362,880 terminal nodes, and only 5,478 distinct legal positions exist. For a non-terminal position the evaluation function is $Eval(s) = 3 \cdot X_2 + X_1 - (3 \cdot O_2 + O_1)$, where X_n is the number of rows, columns or diagonals with exactly n X's and no O's.

(a) Count the distinct legal positions by exhaustive enumeration and confirm the figure of 5,478; compute the number of game-tree nodes with and without exploiting transpositions

(b) Implement depth-limited minimax with Eval for depth limits 1 to 9 and record the nodes examined and the move chosen at the first turn; determine the smallest depth at which the first move is optimal

(c) Design a different evaluation function (for example, one that weights the centre and corners) and compare it with Eval over 100 games against a random opponent and against full-depth minimax

(d) Analyze the position classes at depth 2 (X in the centre, a corner or an edge) and relate them to the backed-up minimax values

Evidence Expected: Correct implementation, enumeration results, depth-versus-node tables, evaluation-function comparison and interpretation.

Task 5: Experiment with Constraint Satisfaction Problems: Derive a Constraint-Propagation Algorithm from a Principle (7 Marks)

CO2 Topic Mapping: Topic 7 – Constraint Satisfaction Problems (CSP)

Problem Context – Map Colouring of Australia (AIMA Section 6.1)

The regions WA, NT, SA, Q, NSW, V and T must each be coloured red, green or blue so that neighbouring regions differ. The borders are WA–NT, WA–SA, NT–SA, NT–Q, SA–Q, SA–NSW, SA–V, Q–NSW and NSW–V; T borders no region.

You are required to develop a constraint-propagation solution for this constraint satisfaction problem (CSP) and to experiment with it together with backtracking search.

(a) Starting only from the principle "a value is deleted from a domain if it has no supporting value in a neighbouring domain", derive an arc-consistency algorithm; write pseudocode and trace it on the partial assignment {WA = green, V = red} to show that it detects the inconsistency (AIMA Exercise 6.11)

(b) Implement plain backtracking, backtracking with forward checking, and backtracking with MRV, degree and least-constraining-value orderings; count the number of solutions with 2, 3 and 4 colours (expect 0, 18 and 768) and the assignments tried by each method

(c) Predict, then measure, the effect of each ordering heuristic on the number of backtracks for n-queens with n = 8, 12, 16

(d) Design one CSP of your own whose constraint graph is a tree and show that it can be solved without backtracking (AIMA Section 6.5)

Evidence Expected: A student-derived propagation rule, pseudocode, implementation, designed test cases, computation counts and interpretation.

Task 6: Experiment with Local Search and Optimization Problems versus Exhaustive Search (8 Marks)

CO2 Topic Mapping: Topic 5 – Local Search Algorithms and Optimization Problems

Problem Context – The 8-Queens Problem (AIMA Section 4.1)

The 8-queens problem has 8^8 = 16,777,216 states if each queen may be anywhere in its column, and 8! = 40,320 states if queens are placed in distinct rows. AIMA reports that steepest-ascent hill climbing from a random state solves only 14% of instances (about 4 steps when it succeeds and 3 when it gets stuck), that allowing up to 100 consecutive sideways moves raises the success to 94% (about 21 steps on success, 64 on failure), and that random-restart hill climbing needs about 1/p restarts.

(a) Design an exhaustive enumeration that finds all solutions; confirm 92 solutions (12 distinct up to symmetry) and record the number of states examined

(b) Implement steepest-ascent hill climbing, hill climbing with sideways moves, random-restart hill climbing and simulated annealing; predict the success rates and average steps, then measure them over 1,000 random starts and compare with the figures reported in the book

(c) Compute the expected number of restarts and steps of random-restart hill climbing from your measured success probability p

(d) Scale the problem to n = 16, 32, 64: determine where exhaustive search becomes infeasible and how the local-search cost grows

Evidence Expected: Student-designed exhaustive enumeration, correctness verification, comparison with the book's figures, growth prediction and validation.

DAY-4 DESIGN CHALLENGE

Task 7: Diagnose, Redesign and Validate: Solving Complex AI Problems Efficiently (13 Marks)

CO2 Topic Mapping: Any of Topics 1–9, depending on the solution selected from Tasks 1–6

Problem Context – The n-Queens Solver That Does Not Scale

A colleague's n-queens solver places queens column by column with plain backtracking and no ordering heuristic. It solves n = 8 quickly, but its running time grows sharply with n. AIMA Section 6.4 reports that the min-conflicts heuristic solves even the million-queens problem in an average of about 50 steps after the initial assignment.

Objective: the redesign must help solve a complex AI problem more efficiently, and the improvement must be demonstrated with a measurable operation count (nodes expanded, constraint checks or evaluation calls).

A. Diagnose the Computational Bottleneck – 2 Marks

Using your own measurements for n = 8, 10, 12, 14, 16, determine where most computational effort is spent and why it grows so quickly.

B. Set the Design Objective – 2 Marks

Decide what your redesign must achieve (for example, solve n = 100 within one second) and set a measurable objective.

C. Design the Revised Computational Strategy – 4 Marks

Analyze what you have changed: the property your new design exploits (for example, forward checking, MRV ordering, or min-conflicts local search), the operations avoided, whether extra memory is introduced and the new trade-offs created. Provide pseudocode.

D. Predict Before Implementing – 1 Mark

Predict the operation counts of the original and the redesigned solver for n = 8 and n = 100 before running anything.

E. Implement and Validate the Redesign – 2 Marks

Implement the redesign, verify every returned board, and compare the measured counts with your prediction.

F. Defend the Final Design – 2 Marks

Defend the design with evidence: where does the new strategy fail (for example, plateaus for min-conflicts), and what would you change for a different CSP?

Answers – Assignment 2: Informed Search, Local Search, Games and CSP (CSA17, CO2)

All programs are Python 3 (Jupyter / Colab compatible; standard library plus numpy). Every number below was produced by running the code listed in the appendix. Fixed seeds are used (`random.seed(42)` or a seed stated with the experiment). Predictions were written down before the experiments were run and are compared with the measurements.

Task 1: Greedy best-first search, A* and weighted A* on the Romania map

(a) Prediction and measured run, Arad to Bucharest

Prediction (before running): Greedy expands Arad, Sibiu, Fagaras and then selects Bucharest, giving Arad–Sibiu–Fagaras–Bucharest, 450 km. A* expands Arad, Sibiu, Rimnicu Vilcea, Fagaras, Pitesti and then selects Bucharest, giving the route through Pitesti, 418 km.

Algorithm	Expansion order (goal is the last node selected)	Route	Cost	Matches prediction
Greedy best-first (f = h)	Arad, Sibiu, Fagaras → Bucharest	Arad-Sibiu-Fagaras-Bucharest	450 km	Yes
A* (f = g + h)	Arad, Sibiu, Rimnicu Vilcea, Fagaras, Pitesti → Bucharest	Arad-Sibiu-Rimnicu Vilcea-Pitesti-Bucharest	418 km	Yes

A* node	g(n)	h(n)	f(n) = g + h
Arad	0	366	366
Sibiu	140	253	393
Rimnicu Vilcea	220	193	413
Fagaras	239	176	415
Pitesti	317	100	417
Bucharest (via Pitesti)	418	0	418
Bucharest (via Fagaras, found earlier)	450	0	450 (replaced by the cheaper path)

Greedy selects by h only: after Sibiu (h = 253) it prefers Fagaras (h = 176) over Rimnicu Vilcea (h = 193), and Bucharest is reached by the 211 km road, 32 km longer than the optimum. A* adds the cost already paid, so Fagaras (f = 415) is passed over until the Pitesti route (f = 417) has been tried and the goal is selected at f = 418 < 450.

(b) Admissibility and consistency of h_{SLD}

Proof of admissibility. h(n) is the Euclidean distance from n to Bucharest. Any road path is a curve between the two points, and the straight segment is the shortest curve, so the road distance is never smaller than h(n): h(n) ≤ h*(n) for every city. **Proof of consistency:** for a road n–n′ of length c, the triangle inequality gives |h(n) − h(n′)| ≤ (straight distance n–n′) ≤ c, which is h(n) ≤ c + h(n′) in both directions. The checks below confirm both with the given values.

City	h(n)	h*(n)	h ≤ h*?	City	h(n)	h*(n)	h ≤ h*?
Arad	366	418	Yes	Neamt	234	406	Yes
Craiova	160	239	Yes	Oradea	380	429	Yes
Drobeta	242	359	Yes	Pitesti	100	101	Yes
Eforie	161	269	Yes	Rimnicu Vilcea	193	198	Yes
Fagaras	176	211	Yes	Sibiu	253	278	Yes
Giurgiu	77	90	Yes	Timisoara	329	536	Yes
Hirsova	151	183	Yes	Urziceni	80	85	Yes
Iasi	226	319	Yes	Vaslui	199	227	Yes
Lugoj	244	504	Yes	Zerind	374	493	Yes
Mehadia	241	434	Yes				

All 19 cities satisfy h ≤ h* (smallest slack 1 km at Pitesti: 100 against 101). Consistency on the 23 roads (46 directed inequalities, 0 violations):

Road	c	h(A)	h(B)	|h(A) − h(B)|	Holds both ways?	Slack c − |Δh|
Oradea – Zerind	71	380	374	6	Yes	65
Zerind – Arad	75	374	366	8	Yes	67
Arad – Timisoara	118	366	329	37	Yes	81
Arad – Sibiu	140	366	253	113	Yes	27
Oradea – Sibiu	151	380	253	127	Yes	24
Sibiu – Fagaras	99	253	176	77	Yes	22

Road	c	h(A)	h(B)	|h(A) − h(B)|	Holds both ways?	Slack c − |Δh|
Sibiu – Rimnicu Vilcea	80	253	193	60	Yes	20
Timisoara – Lugoj	111	329	244	85	Yes	26
Lugoj – Mehadia	70	244	241	3	Yes	67
Mehadia – Drobeta	75	241	242	1	Yes	74
Drobeta – Craiova	120	242	160	82	Yes	38
Craiova – Rimnicu Vilcea	146	160	193	33	Yes	113
Craiova – Pitesti	138	160	100	60	Yes	78
Rimnicu Vilcea – Pitesti	97	193	100	93	Yes	4
Pitesti – Bucharest	101	100	0	100	Yes	1
Fagaras – Bucharest	211	176	0	176	Yes	35
Bucharest – Giurgiu	90	0	77	77	Yes	13
Bucharest – Urziceni	85	0	80	80	Yes	5
Urziceni – Hirsova	98	80	151	71	Yes	27
Hirsova – Eforie	86	151	161	10	Yes	76
Urziceni – Vaslui	142	80	199	119	Yes	23
Vaslui – Iasi	92	199	226	27	Yes	65
Iasi – Neamt	87	226	234	8	Yes	79

The tightest roads are Pitesti–Bucharest (slack 1), Rimnicu Vilcea–Pitesti (slack 4), Bucharest–Urziceni (slack 5); h is **admissible and consistent**, so A* is optimal and never needs to re-open a node on the base map.

(c) Behaviour under changed map conditions (Arad to Bucharest; last column over all 19 start cities)

Map condition	h admiss. / consist.	Greedy	A*	Weighted A* (W = 1.5)	UCS (optimal reference)	Greedy / A* optimal on all starts
Base map (AIMA)	Yes / Yes	450 km, 3 exp.	418 km, 5 exp.	450 km, 3 exp.	418 km, 12 exp.	14/19 / 19/19
C1: road Pitesti-Bucharest removed	Yes / Yes	450 km, 3 exp.	450 km, 7 exp.	450 km, 3 exp.	450 km, 12 exp.	17/19 / 19/19
C2a: Fagaras-Bucharest cost 211 -> 700 (flooded)	Yes / Yes	939 km, 3 exp.	418 km, 5 exp.	418 km, 5 exp.	418 km, 12 exp.	13/19 / 19/19
C2b: new tunnel Timisoara-Bucharest, 100 km	No / No	450 km, 3 exp.	418 km, 5 exp.	450 km, 3 exp.	218 km, 5 exp.	12/19 / 14/19
C3: dead end Oasis off Sibiu (30 km, h=40)	Yes / No	450 km, 4 exp.	418 km, 6 exp.	450 km, 4 exp.	418 km, 13 exp.	14/20 / 20/20

- **C1 (road removed).** Without Pitesti–Bucharest the optimum is 450 km; Greedy and A* both return it, so removing the road greedy was tempted away from makes Greedy look as good as A* here.
- **C2a (changed cost).** Making Fagaras–Bucharest a 700 km flooded road leaves h admissible and consistent (costs only increased), so A* still returns 418 km, but Greedy still follows h into Fagaras and pays 939 km (2.25 times the optimum): straight-line distance is no longer a good guide to cost.
- **C2b (tunnel).** A new 100 km tunnel Timisoara–Bucharest (straight distance 329 km) makes h inadmissible: h(Timisoara) = 329 > h* = 100. A* returns 418 km although 218 km (Arad–Timisoara–Bucharest) exists, because Timisoara (f = 118 + 329 = 447) is never expanded before the goal is selected at 418. UCS finds 218 km. Optimality of A* depends on admissibility.
- **C3 (dead end).** A dead-end city Oasis next to Sibiu with a small h (40) costs Greedy and A* one extra expansion each (4 against 3 and 6 against 5), but both still find the right routes; h is inconsistent here (253 > 30 + 40) yet admissible, and A* with node re-opening stays optimal.

Conclusion: no algorithm is best in all conditions. Greedy is fastest but unreliable when h misleads, A* is optimal only while h is admissible, and UCS is the safe fallback when no trustworthy heuristic exists.

(d) Weighted A* (f = g + W·h) on the base map

W	Arad → Bucharest cost	Expansions	Optimal?	All 19 starts: optimal	Avg expansions	Mean cost / optimal	Worst ratio
1	418 km	5	Yes	19/19	3.53	1.0	1.0
1.2	450 km	3	No	14/19	3	1.0206	1.115
1.5	450 km	3	No	14/19	2.79	1.0252	1.147
2	450 km	3	No	14/19	2.79	1.0252	1.147
3	450 km	3	No	14/19	2.74	1.0252	1.147

Where optimality is lost: scanning W in steps of 0.01, Arad → Bucharest stays optimal up to W = 1.19 and returns 450 km from W = 1.20. Exactly: the goal reached through Fagaras is selected at f = 450, while Rimnicu Vilcea has f = 220 + 193W; the Fagaras route wins once 450 < 220 + 193W, i.e. W > 230/193 = 1.192. Over all 19 start cities the first loss is also at W = 1.20 (Oradea, 461 km against 429 km). Larger W cuts the average number of expansions from 3.53 (W = 1) to 2.74 (W = 3), at the price of a mean cost 2.1% to 2.5% above the optimum (worst start 11.5% to 14.7% above); weighted A* guarantees a cost within W times the optimum, so W between 1.0 and 1.5 is a sensible trade on this map.

(e) A* against uniform-cost search on the same 19 pairs (start city to Bucharest)

Start	A* cost	UCS cost	A* expansions	UCS expansions	Saved
Arad	418	418	5	12	7
Craiova	239	239	2	6	4
Drobeta	359	359	4	8	4
Eforie	269	269	3	3	0
Fagaras	211	211	1	3	2
Giurgiu	90	90	1	1	0
Hirsova	183	183	2	3	1
Iasi	319	319	3	4	1
Lugoj	504	504	6	12	6
Mehadia	434	434	5	10	5
Neamt	406	406	4	4	0
Oradea	429	429	5	10	5
Pitesti	101	101	1	2	1
Rimnicu Vilcea	198	198	2	5	3
Sibiu	278	278	4	9	5
Timisoara	536	536	9	12	3
Urziceni	85	85	1	1	0
Vaslui	227	227	2	4	2
Zerind	493	493	7	12	5
Total			**67**	**121**	**54**

A* returns exactly the UCS (optimal) cost for every pair and uses **67 expansions against 121** in total, saving 54 (44.6%). The saving comes from the heuristic steering the search towards Bucharest, whereas UCS expands in all directions at equal path cost; it is largest for the western cities (Arad 7, Lugoj 6, Zerind, Mehadia and Oradea 5 each).

Task 2: Online search agents (online DFS and LRTA*) on the Romania map

The agent does not know the map. In each city it only learns which roads leave it, and it learns where a road leads, and its length, by driving along it. Goal: Bucharest. **Online DFS** (AIMA Fig. 4.21) keeps *result*, *untried* and *unbacktracked* tables and backtracks along the road it came by. **LRTA*** (AIMA Fig. 4.24) keeps a table H of learned cost-to-go estimates, initialised with h_{SLD}; an untried road is assumed to cost h(s), a tried road costs c + H(next). Neighbours are tried in alphabetical order, ties broken alphabetically. The **competitive ratio** is the distance driven divided by the optimal distance a map-knowing agent would drive.

(a) Known reversible roads: every start city to Bucharest

Start	Optimal km	Online DFS km	Ratio	Steps	LRTA* km	Ratio	Steps
Arad	418	1273	3.05	11	1116	2.67	9
Craiova	239	479	2.0	4	479	2.0	4
Drobeta	359	2399	6.68	23	749	2.09	7
Eforie	269	441	1.64	5	441	1.64	5
Fagaras	211	211	1.0	1	211	1.0	1
Giurgiu	90	90	1.0	1	90	1.0	1
Hirsova	183	355	1.94	4	355	1.94	4
Iasi	319	677	2.12	7	677	2.12	7
Lugoj	504	2254	4.47	21	1772	3.52	17
Mehadia	434	2324	5.35	22	1842	4.24	18
Neamt	406	764	1.88	8	764	1.88	8
Oradea	429	741	1.73	5	741	1.73	5
Pitesti	101	101	1.0	1	101	1.0	1
Rimnicu Vilcea	198	625	3.16	5	625	3.16	5

Start	Optimal km	Online DFS km	Ratio	Steps	LRTA* km	Ratio	Steps
Sibiu	278	590	2.12	4	590	2.12	4
Timisoara	536	1883	3.51	18	1986	3.71	18
Urziceni	85	85	1.0	1	85	1.0	1
Vaslui	227	585	2.58	6	585	2.58	6
Zerind	493	1198	2.43	10	1198	2.43	10
Total / mean	**5,779**	**17,075**	**2.56**		**14,407**	**2.20**	

Online DFS drives 17,075 km in total and LRTA* 14,407 km, against 5,779 km for an agent that knows the map. Mean competitive ratio: online DFS **2.56** (worst 6.68 from Drobeta), LRTA* **2.20** (worst 4.24 from Mehadia). The ratio is worst for the western cities (Drobeta, Mehadia, Timisoara, Lugoj), from which the agent must explore a long chain of roads before it meets the Bucharest side of the map; LRTA* is better in total because its heuristic steers it, but it is not better from every start (for example Timisoara: 3.71 against 3.51).

(b) Two one-way roads added: Iasi → Neamt and Hirsova → Eforie (irreversible actions)

Neamt and Eforie each have only one road, so after the roads become one-way they are traps: an agent that enters cannot return. Online DFS relies on backtracking and cannot undo such a move; LRTA* has no way back either.

Start	Optimal km	Online DFS	Distance before failing / reaching goal	LRTA*	Distance	Why it stops
Eforie	unreachable	FAILS	0	FAILS	0	Online DFS: exhausted. LRTA*: trapped at Eforie: no roads leave it.
Hirsova	183	FAILS	86	FAILS	86	Online DFS: trapped at Eforie: no road back to Hirsova. LRTA*: trapped at Eforie: no roads leave it.
Iasi	319	FAILS	87	FAILS	87	Online DFS: trapped at Neamt: no road back to Iasi. LRTA*: trapped at Neamt: no roads leave it.
Neamt	unreachable	FAILS	0	FAILS	0	Online DFS: exhausted. LRTA*: trapped at Neamt: no roads leave it.
Vaslui	227	FAILS	179	FAILS	179	Online DFS: trapped at Neamt: no road back to Iasi. LRTA*: trapped at Neamt: no roads leave it.

Online DFS succeeded from **14/19** start cities and LRTA* from **14/19** (the other 14 starts behave as in (a)). Five starts fail: Neamt and Eforie are themselves traps (no road leaves them), and from Hirsova, Iasi and Vaslui both agents try the alphabetically first untried road (Eforie, Neamt, Iasi) and are trapped there, because LRTA* gives every untried road the same cost h(s). Irreversible actions break the assumption of both algorithms that the space is *safely explorable* (the goal reachable from every state): online DFS needs reversibility to backtrack, and LRTA* needs it to learn from mistakes.

(c) Repeated trials from Arad: does the agent improve?

Trial	1	2	3	4	5	6	7	8	9	10	11	12
LRTA* (AIMA algorithm), km	1116	1147	418	610	450	450	450	450	450	450	450	450
LRTA* (H also updated on reaching the goal), km	1116	1147	1351	418	418	418	418	418	418	418	418	418
Online DFS (no learning), km	1273	1273	1273	1273	1273	1273	1273	1273	1273	1273	1273	1273

City	Initial h	Learned H (AIMA algorithm)	Learned H (goal update)	True cost to Bucharest
Arad	366	415	418	418
Sibiu	253	275	278	278
Fagaras	176	176	211	211
Rimnicu Vilcea	193	197	198	198
Pitesti	100	100	101	101

Over all 19 start cities with shared memory the total distance per sweep is [8830, 5939, 5939, 5939, 5939] km for the AIMA algorithm and [9850, 5779, 5779, 5779, 5779] km with the goal update, against an optimal 5,779 km. Online DFS repeats the same 1,273 km in every trial because it keeps nothing but the map of the current trial.

- LRTA* improves from 1,116 km to a steady **450 km** in 5 trials; online DFS never improves. From Lugoj the LRTA* trials give 1772, 796, 504, 504, 504, 504 km (online DFS: 2254 km every time).
- **Finding:** the AIMA pseudocode returns *stop* as soon as the goal is perceived, before it updates H of the city it just left. H(Fagaras) therefore stays at 176 (true cost 211), the Fagaras route keeps looking cheaper, and LRTA* converges to 450 km instead of 418 km. Updating H(s) on arrival at the goal (the variant) gives H(Fagaras) = 211 and the learned values equal the true costs, so the agent settles on the optimal 418 km from trial 4 and the sweep total falls to the optimal 5,779 km.

(d) When is online search appropriate?

Online (LRTA* or online DFS): the map is unknown or changes, acting is the only way to learn it, or there is no time for a complete plan; these agents need no up-front model, keep small tables, and LRTA* improves with repetition. **Prefer LRTA* over online DFS** when a heuristic exists and the same task repeats. **Offline search (A*, UCS):** when the map is fully known, actions are cheap to simulate, and optimal routes matter; here A* needs 67 expansions against 121 for UCS (Task 1) and no driving at all. **Neither is safe** when actions are irreversible and the space is not safely explorable (Task 2b); then the agent needs a model of dangerous states or must plan conservatively. Competitive ratios of 2.2 to 2.6 are the price of not knowing the map.

Task 3: Minimax and alpha-beta pruning

(a) The tree of AIMA Fig. 5.2 (leaves 3, 12, 8 | 2, 4, 6 | 14, 5, 2)

Algorithm	Backed-up value of B, C, D	Root value / move	Leaves evaluated	Pruned
Minimax	3, 2, 2	3 / move B	9	none
Alpha-beta (left to right)	3, ≤ 2 (cut), 2	3 / move B	7	Leaves 4 and 6 under C (cut at C after the first leaf 2 ≤ α = 3)

Trace. Root α = −∞, β = +∞. Node B: leaves 3, 12, 8 give 3, so root α = 3. Node C (α = 3, β = +∞): the first leaf 2 ≤ α, so C can be at most 2 and MAX never prefers it: leaves 4 and 6 are pruned. Node D (α = 3): leaves 14 (β = 14), 5 (β = 5, still above α), 2 (value 2); nothing is left to prune. Result: root 3, the same as minimax, with 7 of 9 leaves. If the leaves of D are ordered (2, 5, 14) only 5 leaves are needed (3 + 1 + 1); if those of C are ordered (6, 4, 2) all 9 are needed.

(b) Leaves evaluated on complete trees: prediction and measurement

Prediction (before running): best-case ordering $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$ leaves; worst case b^d (no pruning); random ordering about $b^{3d/4}$.
Method: complete trees with b children per node and depth d (d plies, root = MAX), leaf utilities drawn at random (seeds 42·1000 + 10b + d). Best case: at every node the children are sorted so that the best move for the player to move comes first (MAX descending, MIN ascending). Random case: the mean of 30 random trees. Worst case: a constructed tree whose children are ordered so that no cut-off can occur. "Worst first" is the same tree with every node sorted worst-first.

b	d	b^d (minimax)	Best case measured	Predicted $b^{\lceil d/2 \rceil}+b^{\lfloor d/2 \rfloor}-1$	$b^{d/2}$	Random measured (30 trees)	$b^{3d/4}$	Fitted exponent (leaves = $b^{x \cdot d}$)	Worst case measured	Worst-first reorder
2	4	16	7	7	4.0	11.5	8.0	0.88	16	15
2	6	64	15	15	8.0	36.5	22.6	0.865	64	58
2	8	256	31	31	16.0	106.1	64.0	0.841	256	232
3	4	81	17	17	9.0	43.0	27.0	0.856	81	76
3	6	729	53	53	27.0	242	140.3	0.833	729	666
3	8	6561	161	161	81.0	1305.2	729.0	0.816	6561	5125
4	4	256	31	31	16.0	105.7	64.0	0.841	256	238
4	6	4096	127	127	64.0	944.6	512.0	0.824	4096	3515
4	8	65536	511	511	256.0	7439.7	4096.0	0.804	65536	48241

Findings. The best case matches the formula exactly in all 9 settings (for example b = 3, d = 6: 53 of 729 leaves, 7.3%), so alpha-beta searches to about **twice the depth** in the same time, $b^{d/2}$. A constructed worst case evaluates all b^d leaves with no cut-off. Merely putting the worst moves first still prunes a little (666 of 729 for b = 3, d = 6), because cut-offs also come from bounds inherited from higher levels. For random order the fitted exponent is 0.83 for b = 3, d = 6, and decreases with depth towards the textbook 0.75: $b^{3d/4}$ is an asymptotic approximation for large b and underestimates for small b (140 predicted against 242 measured at b = 3, d = 6).

(c) Comparison of the three growth rates

For b = 4, d = 8 minimax evaluates 65,536 leaves; alpha-beta needs 511 with perfect ordering (0.8%, the $b^{d/2}$ = 256 growth), about 7,440 with random ordering (exponent 0.804), and 65,536 in the constructed worst case. For b = 2, d = 8 the numbers are 256, 31, 106.1 and 256. Pruning therefore never changes the value, only the work: from $\mathbf{b^d}$ (worst) through about $\mathbf{b^{0.8d}}$ (random) down to $\mathbf{b^{d/2}}$ (best).

(d) Where the cut-offs occur (b = 3, d = 6, mean of 30 random trees; root depth 0)

Depth of the cutting node	α-cut-offs (at MIN nodes)	β-cut-offs (at MAX nodes)	Share of all pruned leaves
1	1.0	–	29.4%
2	–	1.6	14.4%
3	8.3	–	25.5%
4	–	14.6	14.2%
5	49.9	–	16.4%

Cut-offs at MIN nodes (α-cuts) occur at odd depths and those at MAX nodes (β-cuts) at even depths. By **count** most cut-offs happen at the deepest levels (depth 5: about 50 α-cuts per tree, depth 4: about 15 β-cuts), where each cut skips only one or two leaves; by **leaves saved** the shallow cuts matter most: the single cut-off at depth 1 saves about 29% of the pruned leaves (24% for b = 4, d = 8), because each skips a whole subtree. For b = 4, d = 8 the depth 1–3 cut-offs account for about 63% of the pruned leaves. This is why good move

ordering near the root is the most valuable.

Task 4: Tic-tac-toe – game tree, depth-limited minimax with Eval, and position classes

(a) Size of the game tree (exhaustive enumeration by program)

Quantity	Value	Comment
Complete games = terminal nodes of the game tree	255,168	X wins 131,184; O wins 77,904; draws 46,080. Fewer than 9! = 362,880 because games stop at a win.
Nodes of the game tree (every move sequence)	549,946	Each move sequence counted separately, as a plain minimax would visit them
Distinct legal positions (transpositions merged)	5,478	The same position is reached by many move orders: 549,946 / 5,478 ≈ 100 repeats
Distinct positions up to rotation and reflection	765	8 symmetries of the board

Distinct positions by number of marks (0 to 9): 1, 9, 72, 252, 756, 1,260, 1,520, 1,140, 390, 78. The 9! bound would give 362,880 leaves; the real tree has 255,168 because games end early when a player wins.

(b) Depth-limited minimax with Eval(s) = $3 \cdot X_2 + X_1 - (3 \cdot O_2 + O_1)$

X_n (O_n) is the number of lines holding exactly n X (O) marks and no opposing mark. X is MAX and moves first; terminal wins score ±1000, draws 0; the search counts every node examined (no pruning, no transposition table); ties between equal moves are broken by cell number (0 to 8, row by row).

Depth limit	Nodes examined	Backed-up values of the 9 first moves (cells 0–8)	Move chosen (cell)	True game value of the chosen move
1	10	3, 2, 3, 2, 4, 2, 3, 2, 3	4	0 (draw)
2	82	-1, -2, -1, -2, 1, -2, -1, -2, -1	4	0 (draw)
3	586	3, 2, 3, 2, 5, 2, 3, 2, 3	4	0 (draw)
4	3,610	-2, -3, -2, -3, 0, -3, -2, -3, -2	4	0 (draw)
5	18,730	3, 2, 3, 2, 5, 2, 3, 2, 3	4	0 (draw)
6	73,450	-1, -3, -1, -3, 0, -3, -1, -3, -1	4	0 (draw)
7	221,626	3, 3, 3, 3, 4, 3, 3, 3, 3	4	0 (draw)
8	422,074	0, 0, 0, 0, 0, 0, 0, 0, 0	0	0 (draw)
9	549,946	0, 0, 0, 0, 0, 0, 0, 0, 0	0	0 (draw)

Result. Every first move has game value 0, so with perfect play from both sides X can draw wherever it starts; every move is "optimal" in value. With depth 1 to 7 Eval selects the **centre** (cell 4), and the choice is therefore already value-optimal at depth 1. From depth 8 the backed-up values equal the true values (all 0), Eval no longer influences anything, and the tie-break selects cell 0 (a corner). The node count grows quickly at first (× 8.2, 7.1, 6.2 and 5.2 over the first four plies) and then flattens as games end, reaching 549,946 nodes at depth 9.

(c) A second evaluation function

Eval2(s) = (weighted occupancy) + $4 \cdot (X_2 - O_2)$, with cell weights centre 3, corners 2, edges 1 (the number of lines through a cell, scaled), counted +1 for X and −1 for O. It rewards the centre and corners and weights immediate threats more than Eval. Experiment: the agent plays 100 games per setting (50 as X, 50 as O; seed 42; ties broken randomly) against a random player and against a perfect (full-depth minimax) player.

Evaluation / depth limit	vs random: W / D / L	vs perfect minimax: W / D / L
Eval (task), depth 2	97 / 3 / 0	0 / 100 / 0
Eval (task), depth 4	90 / 10 / 0	0 / 100 / 0
Eval2 (centre/corner weights), depth 2	97 / 3 / 0	0 / 100 / 0
Eval2 (centre/corner weights), depth 4	92 / 8 / 0	0 / 100 / 0

Interpretation: both functions never lose to a random player and always draw against the perfect player (W/D/L 0/100/0), which is the best possible result. The two are nearly equal: at depth 2 both win 97 of 100 games, and at depth 4 Eval2 wins 92 against 90. The extra weighting gives a small improvement at depth 4 (two more wins) that is within the sampling noise of 100 games. A deeper search did *not* win more often against a random opponent (90 and 92 against 97), it draws more often (10 and 8 draws against 3), so a deeper horizon is not automatically better when Eval is the only guide at the leaves.

(d) Position classes after two plies (X moves, O replies)

Class of X first move	Positions after 2 plies	O replies that still draw	O replies that lose (X wins)	O replies that win	Depth-2 backed-up Eval of the first move (min over replies)
centre	8	4	4	0	1
corner	32	4	28	0	-1
edge	32	16	16	0	-2

There are 72 positions after two plies: 8 after a centre opening, 32 after a corner opening (4 corners × 8 replies) and 32 after an edge opening. True values from full minimax: after a **centre** opening 4 of the 8 replies draw (the corners) and 4 lose (the edges); after a **corner** opening only 1 of 8 replies draws per corner (the centre) and the other 7 lose, so 28 of 32 positions are won for X; after an **edge** opening 4 of 8 replies draw per move (16 of 32) and 16 are won for X. O can never win from any of these positions, since X cannot lose by playing any first move.

Relation to the depth-2 values: the depth-2 backed-up Eval ranks the centre highest (+1), then the corner (−1), then the edge (−2), while the true values are all 0 (draw). Eval agrees with the true ordering on the weakest opening (the edge, which gives O the most drawing replies) but not on the strongest *practical* opening: the corner gives O only one drawing reply out of 8 (87.5% of replies lose) against 50% for the centre. Eval at depth 2 therefore measures positional features, not the likelihood that the opponent errs; this explains why the choice of the centre by Eval is safe (value 0) but not the opening that maximises winning chances against a fallible opponent.

Task 5: Constraint satisfaction – AC-3, backtracking heuristics, n-queens and a tree-structured CSP

(a) AC-3: derivation and trace for the partial assignment {WA = green, V = red}

Derivation. A value x of X is acceptable in a binary constraint with Y only if some value y in D(Y) supports it (here x ≠ y). Deleting an unsupported value can remove the support of a neighbour value, so every arc (Z, X) of the neighbours Z of X must be re-checked. AC-3 keeps a queue of arcs, revises each arc, and re-queues the arcs pointing at X whenever D(X) shrinks; if a domain becomes empty the problem is inconsistent.

```
AC-3(domains, arcs):                          REVISE(X, Y):
  queue <- all arcs (X, Y)                      removed <- false
  while queue not empty:                        for x in D[X]:
     (X, Y) <- pop                                if no y in D[Y] with x != y:   # no support
     if REVISE(X, Y):                                 delete x from D[X]; removed <- true
        if D[X] is empty: return FAILURE         return removed
        for each neighbour Z != Y of X: push (Z, X)
  return CONSISTENT
```

Step	Revise(X, Y)	Values removed	Domain D(X) afterwards
1	(NSW, V)	red	{blue, green}
2	(NT, WA)	green	{blue, red}
3	(SA, V)	red	{blue, green}
4	(SA, WA)	green	{blue}
5	(NSW, SA)	blue	{green}
6	(NT, SA)	blue	{red}
7	(Q, SA)	blue	{green, red}
8	(Q, NSW)	green	{red}
9	(Q, NT)	red	{ } (EMPTY)

Start: D(WA) = {green}, D(V) = {red}, every other domain {red, green, blue}. AC-3 processed 33 arcs, the nine listed revisions removed values, and the domain of Q became **empty**, so arc consistency alone proves that {WA = green, V = red} cannot be extended to a solution, without any search. Reason: WA = green and V = red force SA = blue, which forces NT = red and NSW = green, leaving Q neither red (NT), blue (SA) nor green (NSW).

(b) Backtracking variants on the Australia map: counts of assignments tried

Assignments tried = value assignments attempted; **backtracks** = failed subtrees. Variables in the order WA, NT, SA, Q, NSW, V, T; values in the order red, green, blue (yellow). Variants: plain backtracking, backtracking with forward checking (FC), and FC with MRV, degree and least-constraining-value (LCV) orderings.

Colours k	Solutions found	Plain: assigned / back (all solutions)	FC: assigned / back (all solutions)	FC+MRV+deg+LCV: assigned / back (all)	First solution: plain	First solution: FC	First solution: FC+MRV+deg+LCV
2	0	10 / 5	4 / 3	4 / 3	10 / 5	4 / 3	4 / 3
3	18	102 / 34	51 / 34	51 / 34	11 / 0	7 / 0	7 / 0
4	768	1508 / 377	1144 / 377	1144 / 377	11 / 0	7 / 0	7 / 0

The solution counts are **0, 18 and 768** for 2, 3 and 4 colours (the textbook values), so every variant is complete and correct. Forward checking halves the assignments for k = 3 (102 → 51) and cuts k = 4 by 24%, and for the first solution it needs 7 instead of 11. When *all* solutions are enumerated, the whole tree must be searched whatever the order, so MRV, degree and LCV change nothing (identical counts to FC); their benefit shows only when the search stops at the first solution or when failures occur (next experiment). For k = 2 the 7 regions with the triangle WA–NT–SA cannot be 2-coloured: FC detects this after 4 assignments (3 backtracks) instead of 10 for plain backtracking.

(c) n-queens: effect of each ordering heuristic (first solution; variable = column, value = row)

Prediction (before running): forward checking cuts backtracks by orders of magnitude; MRV improves on FC again; LCV helps less because it only reorders values; the degree heuristic has little effect because every pair of columns constrains each other, so all degrees are equal.

Method	n = 8: assigned / backtracks	n = 12: assigned / backtracks	n = 16: assigned / backtracks
Plain backtracking	876 / 105	3,066 / 249	160,712 / 10,036
FC	88 / 45	193 / 99	7,560 / 4,362
FC + MRV	75 / 44	153 / 95	44 / 21
FC + MRV + degree	75 / 44	153 / 95	44 / 21
FC + MRV + LCV	39 / 21	32 / 13	332 / 223
FC + MRV + degree + LCV	39 / 21	32 / 13	332 / 223

- **Forward checking** cuts backtracks from 10,036 to 4,362 at n = 16 (2.3×), 249 to 99 at n = 12 and 105 to 45 at n = 8; the prediction of "orders of magnitude" was **too optimistic** for FC alone.
- **MRV** then gives the big gain at n = 16: 44 assignments and 21 backtracks, 3,650× fewer assignments than plain backtracking (160,712), because it picks the column with the fewest legal rows and fails early.
- **LCV** helped at n = 8 and 12 (39 and 32 assignments against 75 and 153 for FC+MRV) but **hurt at n = 16** (332 against 44): choosing the row that rules out the fewest values is not always the one that leads to a solution quickly; a lucky ordering matters more than the heuristic. The prediction that LCV helps less than MRV held at n = 16 only.
- **Degree** changed nothing in any case, as predicted: every unassigned column constrains all others.

(d) A tree-structured CSP and its linear-time solution

Problem (own design): exam sequencing. Seven exams form a prerequisite tree: Maths → {Physics, Programming}; Physics → {Mechanics, Optics}; Programming → {Algorithms, Databases}. Each exam gets a slot in {1, 2, 3, 4}; a prerequisite must be strictly earlier than the exam depending on it (slot(parent) < slot(child)). Unary limits: Algorithms only in slot 4, Databases in slot 2 or 3. The constraint graph is a tree.

Algorithm (AIMA 6.5). (1) Pick a root and order the variables so every parent comes before its children. (2) For j = n down to 2 make the arc (parent(X_j), X_j) consistent by deleting from the parent's domain every value without support in the child's domain. (3) For j = 1 to n assign X_j any value consistent with its parent. After step 2 every value that remains has a supporting value in each child, so step 3 never fails: **no backtracking**, time $O(n \cdot d^2)$ for n variables and domain size d.

Step 2: arc (parent, child) made consistent	Parent domain before	Parent domain after
(Programming, Databases)	{1, 2, 3, 4}	{1, 2}
(Programming, Algorithms)	{1, 2}	{1, 2}
(Physics, Optics)	{1, 2, 3, 4}	{1, 2, 3}
(Physics, Mechanics)	{1, 2, 3}	{1, 2, 3}
(Maths, Programming)	{1, 2, 3, 4}	{1}
(Maths, Physics)	{1}	{1}

Step 3 assigns, in order: Maths = 1, Physics = 2, Programming = 2, Mechanics = 3, Optics = 3, Algorithms = 4, Databases = 3. Backtracks used: **0**. Check: Maths 1 < Physics 2 < Mechanics 3 and Optics 3; Maths 1 < Programming 2 < Algorithms 4 and Databases 3 (in {2,3}). All 6 precedence constraints and both unary limits hold.

Plain backtracking on the same CSP, variable order	Assignments tried	Backtracks
parents first (Maths, Physics, ...)	14	0
alphabetical (leaves first)	426	103
reverse (leaves to root)	395	96
Tree algorithm (above)	**7 assignments + 6 arc revisions**	**0**

Plain backtracking is sensitive to the variable order: with parents first it also finds the solution without backtracking (14 assignments) but with the leaves first (alphabetical order, or reverse) it makes about 400 assignments and 96–103 backtracks, because it assigns children before their parents are known and discovers conflicts late. The tree algorithm is order-independent in cost and always backtrack-free.

Task 6: 8-queens by local search

(a) Exhaustive enumeration

State space: one queen per column, any row: 8^8 = 16,777,216 states. Restricting to one queen per row *and* column leaves the permutations of the rows: **40,320 states examined**, each checked for diagonal attacks, giving **92 solutions**. Grouping the solutions under the 8 rotations and reflections of the board gives **12 distinct solutions**, as expected. A column-by-column backtracking search that places a queen only on a safe square finds the same 92 solutions after examining only **2,057 partial placements**.

(b) Hill climbing, sideways moves, annealing: prediction and measurement

State and heuristic: one queen per column; h = number of attacking pairs; a move relocates one queen within its column (8 × 7 = 56 neighbours). **Steepest-ascent:** move to the neighbour with the lowest h, stop when none is strictly better. **Sideways:** also accept equal-h neighbours, at most 100 in a row. **Simulated annealing:** pick a random neighbour; accept if h does not increase, otherwise with probability $e^{-\Delta h/T}$; T starts at 2 and is multiplied by 0.995 each iteration (cap 200,000). **Experiment:** 1,000 random starts (seed 42) for each; "steps" are moves made; the textbook values (AIMA 4.1.1) are shown for comparison.

Algorithm	Predicted success	Measured success	Steps when successful	Steps when stuck	Textbook (AIMA)
Steepest-ascent	≈ 14%	13.4%	4.1	3.1	14%; 4 / 3 steps
Hill climbing with sideways moves (limit 100)	≈ 94%	94.0%	18.9	63.3	94%; 21 / 64 steps
Simulated annealing (T0 = 2, factor 0.995)	not predicted	97.7%	977 iterations	–	–

The predictions hold: steepest-ascent solves only 13.4% of random starts and gets stuck after about 3 moves, because a local minimum is reached almost at once; allowing sideways moves lifts success to 94.0% at the price of longer runs (about 19 moves, and 63 moves before giving up). Simulated annealing succeeds in 97.7% of runs but needs about 977 iterations, each iteration being a single cheap move evaluation.

(c) Random restarts: expected number of runs and steps

If one run succeeds with probability p, the number of runs until the first success is geometric with mean **1/p**, i.e. (1 − p)/p failed runs. Expected total steps = (1 − p)/p × (steps of a failed run) + (steps of the successful run).

Variant	p	Expected runs 1/p	Expected steps (formula)	Measured over 1,000 trials: runs	Measured steps
Steepest-ascent	0.134	7.46	6.46 × 3.1 + 4.1 = 24.1	7.0	22.3
Sideways (limit 100)	0.94	1.06	0.064 × 63.3 + 18.9 = 22.9	1.06	22.9

Theory and measurement agree (7.5 against 7.0 runs; 24.1 against 22.3 steps; 1.06 against 1.06 runs; 22.9 against 22.9 steps). Both variants need about **22 steps** in total, but the steepest version spends them over about 7 restarts, while sideways moves solve the problem in a single run almost every time (a restart is needed in only about 6% of the runs).

(d) Scaling to n = 16, 32 and 64

Exhaustive search. The number of states is n! (one queen per row and column): $16! = 2.1 \times 10^{13}$, $32! = 2.6 \times 10^{35}$, $64! = 1.3 \times 10^{89}$. Even the pruned column-by-column backtracking that lists every solution (measured, bitmask implementation, partial placements examined):

n	8	9	10	11	12	13	14	15
Solutions	92	352	724	2,680	14,200	73,712	365,596	2,279,184
Nodes examined	2,057	8,394	35,539	166,926	856,189	4,674,890	27,358,553	171,129,072
Time (Python), s	0.0	0.0	0.01	0.05	0.26	1.57	9.82	56.78

The nodes grow by a factor of about **5.05 per extra queen** (log-linear fit, n = 8 to 15). Extrapolating: n = 16 needs about 6.7e+08 nodes, n = 20 about 4.3e+11, n = 24 about 2.8e+14, n = 32 about 1.2e+20 and n = 64 about 3.8e+42. At the measured Python rate of 3.0e+06 nodes/s, enumeration stops being feasible (more than one hour) at **n = 18**; with an optimised program at 10^8 nodes/s the limit is **n = 20**. Listing all solutions is hopeless beyond that, although finding *one* solution is easy (Task 7).

n	Algorithm	Success rate	Steps / iterations when successful	Neighbour evaluations per run	Time per run
16	Steepest-ascent	2%	6.5	1,814	0.0 s
16	Sideways (100)	96%	30.9	8,191	0.001 s
16	Simulated annealing	100%	3,759 iterations	1 per iteration	0.008 s
32	Steepest-ascent	1%	16	14,751	0.0 s
32	Sideways (100)	100%	37.5	37,190	0.001 s
32	Simulated annealing	100%	11,586 iterations	1 per iteration	0.027 s
64	Steepest-ascent	2%	31	118,541	0.002 s
64	Sideways (100)	100%	55.5	223,910	0.003 s
64	Simulated annealing	98%	34,084 iterations	1 per iteration	0.085 s

Local search scales gently: with sideways moves the steps to a solution grow about linearly (31, 38 and 56 moves for n = 16, 32 and 64) and the work per run stays below a few milliseconds, whereas plain steepest-ascent collapses (2%, 1% and 1.7% success) because the plateau-free uphill path almost never ends at a solution. Annealing succeeds in 98 to 100% of runs, with iterations growing roughly as $n^{1.6}$ (3,759; 11,586; 34,084). Runs per setting: 100 (n = 16, 32), 60 (n = 64).

Task 7: Diagnose, redesign and validate an n-queens solver

The existing solver (a colleague's design) places the queens column by column, trying rows in ascending order, and checks each new queen against all queens already placed (plain backtracking). It is correct but must handle n up to about 1,000 in a scheduling

application.

A. Diagnosis: where does the effort go?

n	Assignments tried	Constraint checks	Backtracks	Checks per assignment	Time (s)	Growth of assignments vs previous n
8	876	2,438	105	2.78	0.001	–
10	975	3,172	92	3.25	0.001	× 1.1
12	3,066	11,755	249	3.83	0.003	× 3.1
14	26,495	119,080	1,885	4.49	0.023	× 8.6
16	160,712	827,932	10,036	5.15	0.122	× 6.1

Findings (n = 16). (1) Nearly all the work is done in the middle columns: columns 8 to 13 receive 94.8% of the 160,712 assignments (column 11 alone 42,071), where the early choices made in columns 0–7 are finally found to be unworkable, i.e. **thrashing**: the search repeatedly discovers the same failure after many levels. (2) Each assignment is checked against every placed queen: the cost per assignment grows from 2.8 checks (n = 8) to 5.2 (n = 16), so the total checks (827,932) grow faster than the assignments. (3) The number of assignments grows irregularly but strongly with n (from × 1.1 to × 8.6 for each two extra queens) and the solver does not scale: for **n = 100** it was stopped after 3,000,000 assignments (5.9 s) having reached column 68 of 100 without a solution.

B. Objective

Find a valid placement for n = 100 in under 1 s, n = 1,000 in under 5 s and n = 10,000 in under 30 s, with about 100 or fewer moves per solution on average, on a single core in Python; every returned board must pass an independent validity check. (Completeness for tight instances is not required: solutions to n-queens are plentiful.)

C. Redesign: min-conflicts local search with incremental conflict counters

Idea. Do not build a solution incrementally; start with one queen per column (placed greedily on the least-conflicting row) and repeatedly repair it: pick a random conflicted queen and move it, within its column, to the row with the fewest conflicts (ties broken at random). Because n-queens has very many solutions, a good board is always a few repairs away. **Efficiency.** The number of attacks on a square is read in O(1) from three counter arrays (queens per row, per diagonal r − c and per diagonal r + c) that are updated incrementally when a queen moves, instead of comparing with all other queens (O(n)). **Trade-offs.** It uses O(n) memory (counters plus the board) but is *incomplete*: it can stall on a plateau and cannot prove that no solution exists; a step cap with random restart is added.

```
MIN-CONFLICTS(n, cap):                          # one queen per column, row[c] = its row
  for c = 0..n-1: place queen c on a row with minimum conflicts (counters row, d1, d2)
  repeat up to cap steps:
     bad <- columns whose queen is attacked
     if bad is empty: return board
     c <- random column in bad;  remove queen c from the counters
     x <- row with minimum conflicts for column c (ties random);  place queen at (c, x), update counters
  return FAILURE  -> restart with a new board (cap = max(100, 5n))
```

D. Predicted and measured operation counts

Predictions (before running): original solver ≈ 10^3 assignments for n = 8 and more than 10^9 for n = 100; min-conflicts ≈ 10–30 steps for n = 8 and 50–100 steps for n = 100.

n	Original: assignments (constraint checks)	Min-conflicts: steps, mean (conflict evaluations)	Restarts used	Time per solution
8	876 (2,438)	51.0 (487)	46 in 200 runs	original 0.001 s ; min-conflicts 0.0006 s
10	975 (3,172)	89.6 (1,052)	112 in 200 runs	original 0.001 s
12	3,066 (11,755)	91.9 (1,329)	114 in 200 runs	original 0.003 s
14	26,495 (119,080)	76.0 (1,329)	70 in 200 runs	original 0.023 s
16	160,712 (827,932)	65.8 (1,373)	50 in 200 runs	original 0.122 s
24	not run	64.2 (2,211)	33 in 200 runs	–
50	not run	55.4 (5,283)	1 in 200 runs	–
100	> 3,000,000 (stopped after 5.9 s, column 68)	57.0 (15,705)	0 in 100 runs	0.0027 s
1000	not run	47.5 (1,047,550)	0 in 20 runs	0.0324 s
10000	not run	53.4 (100,534,000)	0 in 5 runs	1.3567 s

Prediction check. Original n = 8: 876 assignments (predicted ≈ 10^3, correct). Original n = 100: more than 3×10^6 assignments without finishing (the prediction of more than 10^9 could not be confirmed within my time cap, it is a lower-bound statement only). Min-conflicts n = 100: 57 steps (predicted 50–100, correct); n = 8: 51 steps on average including restarts, above the predicted 10–30: the median single run needs only 24 steps, but about 5% of runs stall and are restarted after 100 steps (see F). The step count is essentially independent of n (47 to 57 from n = 100 to 10,000), which is the key result: the work per solution grows only through the O(n) evaluations per step.

E. Validation

Every board returned by the program was checked by an independent function (distinct rows, distinct r − c and r + c): **200 boards for n = 8, 100 for n = 100, 20 for n = 1,000 and 5 for n = 10,000, all valid** (assertions in the code). Objective met: n = 100 in 0.0027 s (limit 1 s), n = 1,000 in 0.0324 s (limit 5 s), n = 10,000 in 1.3567 s (limit 30 s), mean moves 47 to 57 (limit about 100). Compared with the original solver at n = 16 the redesign uses 1,373 conflict evaluations instead of 827,932 constraint checks (about 603 times fewer).

F. Defence of the redesign and where it fails

- **Plateaus and cycles.** A single run with no restart for n = 8 left 107 of 2000 runs (5.3%) unsolved after 1,000 moves (the first mean-of-runs experiment even had one run still unsolved after 100,000); for n = 16: 8 of 500, for n = 32 and 64: 0 and 0 of 500. The fraction of single runs solved within 10, 25, 50, 100, 200 and 1,000 steps (n = 8) is 30.1%, 48.8%, 64.2%, 79.0%, 89.2% and 94.7%. The remedy is the random restart (cap = max(100, 5n) steps), used in all the measurements above.
- **Incompleteness.** For the unsolvable cases n = 2 and n = 3 the algorithm never succeeds: it runs to the step cap and returns failure (checked for both n = 2 and n = 3: no solution found in 5,000 steps). It cannot tell "no solution" from "not yet found". A complete method is needed when a proof of infeasibility matters.
- **Alternative that stays complete:** backtracking with forward checking and MRV (Task 5) is far better than the original: 43 assignments for n = 24 and 185 for n = 100, but its cost is erratic (n = 50 needed 2,068 assignments and 1,506 backtracks), so I keep min-conflicts as the primary solver and FC+MRV as a complete fallback for small n.
- **Other CSPs.** Min-conflicts works well when solutions are dense, as here. For tightly constrained problems with few solutions (timetables with hard rules, Sudoku) it stalls in local minima; there, constraint propagation with backtracking (Task 5), tree decomposition when the constraint graph is a tree, or simulated annealing with soft constraints are the better choices.

Appendix: source code (Python 3) and how the experiments were run

Files: `core.py` (map), `informed.py` (Task 1), `online.py` (Task 2), `ab.py` (Task 3), `ttt.py` (Task 4), `csp.py` (Task 5), `queens.py` (Tasks 6 and 7). The experiment drivers call these functions with the inputs, seeds and repetition counts stated in each task (for example `random.Random(42)`, 1,000 starts for 8-queens, 30 random trees per (b, d), 100 games per setting), print the tables used above and use `assert` to validate boards, minimax values and solution counts. Running the files again reproduces every number.

core.py

```
"""Romania road map (AIMA Fig. 3.2): 20 cities, 23 two-way roads; helper functions."""
ROADS = [("Oradea","Zerind",71),("Zerind","Arad",75),("Arad","Timisoara",118),("Arad","Sibiu",140),
 ("Oradea","Sibiu",151),("Sibiu","Fagaras",99),("Sibiu","Rimnicu Vilcea",80),("Timisoara","Lugoj",111),
 ("Lugoj","Mehadia",70),("Mehadia","Drobeta",75),("Drobeta","Craiova",120),("Craiova","Rimnicu Vilcea",146),
 ("Craiova","Pitesti",138),("Rimnicu Vilcea","Pitesti",97),("Pitesti","Bucharest",101),("Fagaras","Bucharest",211),
 ("Bucharest","Giurgiu",90),("Bucharest","Urziceni",85),("Urziceni","Hirsova",98),("Hirsova","Eforie",86),
 ("Urziceni","Vaslui",142),("Vaslui","Iasi",92),("Iasi","Neamt",87)]
def build_adjacency(roads):
    adj = {}
    for a, b, km in roads:
        adj.setdefault(a, []).append((b, km)); adj.setdefault(b, []).append((a, km))
    for c in adj: adj[c].sort()
    return adj
```

informed.py

```
"""Task 1: greedy best-first, A* and weighted A* on the Romania map (AIMA Fig. 3.22 heuristic)."""
import heapq
from core import ROADS, build_adjacency
H = {"Arad":366,"Bucharest":0,"Craiova":160,"Drobeta":242,"Eforie":161,"Fagaras":176,"Giurgiu":77,"Hirsova":151,
     "Iasi":226,"Lugoj":244,"Mehadia":241,"Neamt":234,"Oradea":380,"Pitesti":100,"Rimnicu Vilcea":193,"Sibiu":253,
     "Timisoara":329,"Urziceni":80,"Vaslui":199,"Zerind":374}
def best_first(adj, h, start, goal, mode="astar", W=1.0):
    """mode 'greedy': f=h ; 'astar': f=g+W*h ; 'ucs': f=g. Nodes are re-opened when a cheaper path is found.
       Returns (path, cost, expansion order); the goal node itself is not counted as an expansion."""
    def f(g, n): return h[n] if mode == "greedy" else (g if mode == "ucs" else g + W * h[n])
    open_ = [(f(0, start), start, 0)]; best = {start: 0}; parent = {start: None}; order = []; closed = {}
    while open_:
        fv, n, g = heapq.heappop(open_)
        if g > best.get(n, float("inf")) and mode != "greedy": continue
        if goal(n):
            p = []; x = n
            while x is not None: p.append(x); x = parent[x]
            return p[::-1], g, order
        if n in closed and (mode == "greedy" or closed[n] <= g): continue
        closed[n] = g; order.append(n)
        for m, c in adj.get(n, []):
            ng = g + c
            if mode == "greedy":
                if m in closed or m in best: continue
                best[m] = ng; parent[m] = n; heapq.heappush(open_, (f(ng, m), m, ng))
            elif ng < best.get(m, float("inf")):
                best[m] = ng; parent[m] = n; heapq.heappush(open_, (f(ng, m), m, ng))
    return None, None, order
```

online.py

```
"""Task 2: online DFS agent (AIMA Fig. 4.21) and LRTA* agent (AIMA Fig. 4.24) on the Romania map."""
from core import ROADS, build_adjacency
from informed import H, best_first
def make_env(oneway=()):
    adj = build_adjacency(ROADS)
    for a, b in oneway: adj[b] = [(x, k) for x, k in adj[b] if x != a]      # only a -> b is kept
    return adj
def online_dfs(adj, start, goal="Bucharest", max_steps=5000):
    untried, unback, result = {}, {}, {}
    s = a = None; here = start; dist = 0; steps = 0; path = [start]; backtracked = False
    while steps <= max_steps:
        sp = here
        if sp == goal: return dict(ok=True, dist=dist, steps=steps, path=path, why="goal")
```

```
        if sp not in untried: untried[sp] = [b for b, _ in adj[sp]]
        if s is not None:
            result[(s, a)] = sp
            if not backtracked: unback.setdefault(sp, []).insert(0, s)     # only exploration moves create a way back
        backtracked = False
        if not untried[sp]:
            if not unback.get(sp): return dict(ok=False, dist=dist, steps=steps, path=path, why="exhausted")
            target = unback[sp].pop(0)
            if target not in [b for b, _ in adj[sp]]:
                return dict(ok=False, dist=dist, steps=steps, path=path, why=f"trapped at {sp}: no road back to {target}")
            a = target; backtracked = True
        else: a = untried[sp].pop(0)
        km = dict(adj[sp])[a]; dist += km; steps += 1; s = sp; here = a; path.append(a)
    return dict(ok=False, dist=dist, steps=steps, path=path, why="step limit")
def lrta(adj, start, goal="Bucharest", Hmem=None, result=None, max_steps=5000, update_at_goal=False):
    Hm = {} if Hmem is None else Hmem; res = {} if result is None else result
    s = a = None; here = start; dist = 0; steps = 0; path = [start]
    def cost(st, b):
        if (st, b) not in res: return H[st]                                # untried action costs h(s)
        return dict(adj[st])[b] + Hm[res[(st, b)]]
    while steps <= max_steps:
        sp = here
        if sp not in Hm: Hm[sp] = H[sp]
        if sp == goal:
            if update_at_goal and s is not None:
                res[(s, a)] = sp; Hm[s] = min(cost(s, b) for b, _ in adj[s])
            return dict(ok=True, dist=dist, steps=steps, path=path, why="goal")
        if s is not None:
            res[(s, a)] = sp; Hm[s] = min(cost(s, b) for b, _ in adj[s])
        acts = [b for b, _ in adj[sp]]
        if not acts: return dict(ok=False, dist=dist, steps=steps, path=path, why=f"trapped at {sp}: no roads leave it")
        a = min(acts, key=lambda b: (cost(sp, b), b))
        km = dict(adj[sp])[a]; dist += km; steps += 1; s = sp; here = a; path.append(a)
    return dict(ok=False, dist=dist, steps=steps, path=path, why="step limit")
def optimal(adj, s, goal="Bucharest"): return best_first(adj, H, s, lambda x: x == goal, "ucs")[1]
```

ab.py

```
"""Task 3: minimax and alpha-beta pruning on game trees (leaf counts and cut-offs)."""
import random, math
def minimax(node, is_max, cnt):
    if not isinstance(node, list): cnt[0] += 1; return node
    vals = [minimax(c, not is_max, cnt) for c in node]
    return max(vals) if is_max else min(vals)
def tree_height(node):
    h = 0
    while isinstance(node, list): node = node[0]; h += 1
    return h
def alphabeta(node, is_max, alpha, beta, cnt, depth=0, cuts=None, height=None):
    """cnt[0] = leaves evaluated; cuts[...] record cut-offs that really skip at least one child."""
    if not isinstance(node, list): cnt[0] += 1; return node
    if height is None: height = tree_height(node)
    v = -math.inf if is_max else math.inf
    for i, c in enumerate(node):
        r = alphabeta(c, not is_max, alpha, beta, cnt, depth + 1, cuts, height - 1)
        if is_max:
            v = max(v, r)
            if v >= beta:                                  # beta cut-off at a MAX node
                if i < len(node) - 1 and cuts is not None:
                    cuts['beta'][depth] = cuts['beta'].get(depth, 0) + 1
                    cuts['pruned'][depth] = cuts['pruned'].get(depth, 0) + (len(node) - i - 1) * len(node) ** (height - 1)
                return v
            alpha = max(alpha, v)
        else:
            v = min(v, r)
            if v <= alpha:                                 # alpha cut-off at a MIN node
                if i < len(node) - 1 and cuts is not None:
                    cuts['alpha'][depth] = cuts['alpha'].get(depth, 0) + 1
                    cuts['pruned'][depth] = cuts['pruned'].get(depth, 0) + (len(node) - i - 1) * len(node) ** (height - 1)
                return v
            beta = min(beta, v)
    return v
def gen(b, d, rng):
    if d == 0: return rng.random()
    return [gen(b, d - 1, rng) for _ in range(b)]
def value(node, is_max):
    if not isinstance(node, list): return node
    vs = [value(c, not is_max) for c in node]; return max(vs) if is_max else min(vs)
def order(node, is_max, best=True):
    """best=True: best move first (MAX descending, MIN ascending); False: worst move first."""
    if not isinstance(node, list): return node
    kids = [order(c, not is_max, best) for c in node]
    return sorted(kids, key=lambda c: value(c, not is_max), reverse=(is_max if best else (not is_max)))
def worst_tree(b, d, W=None):
    """A tree on which NO cut-off can occur (MAX levels increasing, MIN levels decreasing, lexicographic leaf values)."""
    W = W or (b + 1)
    def rec(depth, acc):
        if depth == d: return float(acc)
        sign = 1 if depth % 2 == 0 else -1
        return [rec(depth + 1, acc + sign * i * W ** (d - depth)) for i in range(b)]
    return rec(0, 0)
def run_ab(tree):
    cnt = [0]; cuts = {'alpha': {}, 'beta': {}, 'pruned': {}}
    v = alphabeta(tree, True, -math.inf, math.inf, cnt, 0, cuts); return v, cnt[0], cuts
def run_mm(tree):
    cnt = [0]; v = minimax(tree, True, cnt); return v, cnt[0]
```

ttt.py

```
"""Task 4: tic-tac-toe -- enumeration, depth-limited minimax with Eval, alternative evaluation, position classes."""
import random, functools
LINES = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]
WIN = 1000
def winner(b):
    for a,c,d in LINES:
        if b[a] != ' ' and b[a] == b[c] == b[d]: return b[a]
    return None
```

```
def moves(b): return [i for i in range(9) if b[i] == ' ']
def play(b, i, p): return b[:i] + p + b[i+1:]
def turn(b): return 'X' if b.count('X') == b.count('O') else 'O'
def terminal(b): return winner(b) is not None or ' ' not in b
def lines_n(b, p, n):
    q = 'O' if p == 'X' else 'X'
    return sum(1 for l in LINES if sum(b[i] == p for i in l) == n and all(b[i] != q for i in l))
def eval1(b):                           # Eval(s) = 3*X2 + X1 - (3*O2 + O1)
    return 3*lines_n(b,'X',2) + lines_n(b,'X',1) - (3*lines_n(b,'O',2) + lines_n(b,'O',1))
W_POS = [2,1,2, 1,3,1, 2,1,2]           # centre 3, corners 2, edges 1
def eval2(b):                           # alternative: centre/corner weights + two-in-a-row threats
    pos = sum(W_POS[i] * (1 if b[i] == 'X' else -1 if b[i] == 'O' else 0) for i in range(9))
    return pos + 4*(lines_n(b,'X',2) - lines_n(b,'O',2))
def utility(b):
    w = winner(b); return WIN if w == 'X' else -WIN if w == 'O' else 0
def mm(b, depth, ev, cnt):
    cnt[0] += 1
    if terminal(b): return utility(b)
    if depth == 0: return ev(b)
    vals = [mm(play(b, i, turn(b)), depth - 1, ev, cnt) for i in moves(b)]
    return max(vals) if turn(b) == 'X' else min(vals)
@functools.lru_cache(maxsize=None)
def full(b):
    if terminal(b): w = winner(b); return 1 if w == 'X' else -1 if w == 'O' else 0
    v = [full(play(b, i, turn(b))) for i in moves(b)]
    return max(v) if turn(b) == 'X' else min(v)
def best_moves(b, depth, ev):
    cnt = [0]; vals = {}
    for i in moves(b): vals[i] = mm(play(b, i, turn(b)), depth - 1, ev, cnt)
    t = turn(b); best = (max if t == 'X' else min)(vals.values())
    return [i for i in vals if vals[i] == best], vals, cnt[0] + 1
def perfect_moves(b):
    t = turn(b); vals = {i: full(play(b, i, t)) for i in moves(b)}
    best = (max if t == 'X' else min)(vals.values()); return [i for i in vals if vals[i] == best]
def game(px, po):
    b = ' ' * 9
    while not terminal(b): b = play(b, (px if turn(b) == 'X' else po)(b), turn(b))
    return winner(b) or 'D'
def sym_forms(b):
    res = []; rot = lambda t: [t[6],t[3],t[0],t[7],t[4],t[1],t[8],t[5],t[2]]; flip = lambda t: [t[2],t[1],t[0],t[5],t[4],t[3],t[8],t[7],t[6]]
    t = list(b)
    for _ in range(4): t = rot(t); res.append(''.join(t)); res.append(''.join(flip(t)))
    return res
```

csp.py

```
"""Task 5: CSP -- AC-3 derived from the support principle, backtracking variants, n-queens, tree CSP."""
from collections import deque
REGIONS = ['WA','NT','SA','Q','NSW','V','T']
BORDERS = [('WA','NT'),('WA','SA'),('NT','SA'),('NT','Q'),('SA','Q'),('SA','NSW'),('SA','V'),('Q','NSW'),('NSW','V')]
NB = {r: [] for r in REGIONS}
for a, b in BORDERS: NB[a].append(b); NB[b].append(a)
def revise(D, X, Y, trace=None):
    """Delete x from D[X] if no y in D[Y] supports it (x != y)."""
    removed = []
    for x in sorted(D[X]):
        if not any(x != y for y in D[Y]): D[X].discard(x); removed.append(x)
    if trace is not None: trace.append((X, Y, removed, {k: sorted(v) for k, v in D.items()}))
    return bool(removed)
def ac3(D, nb=NB, trace=None):
    queue = deque((X, Y) for X in sorted(nb) for Y in sorted(nb[X])); checks = 0
    while queue:
        X, Y = queue.popleft(); checks += 1
        if revise(D, X, Y, trace):
            if not D[X]: return False, checks
            for Z in sorted(nb[X]):
                if Z != Y: queue.append((Z, X))
    return True, checks
class Stats:
    def __init__(self): self.assign = 0; self.back = 0; self.solutions = 0
def solve_csp(vars_, dom, nb, ok, method, all_solutions=False, limit=None):
    """method: plain | fc | fc_mrv | fc_mrv_deg | fc_mrv_lcv | fc_mrv_deg_lcv.
       assign = value assignments tried; back = dead-end calls (failed subtrees)."""
    st = Stats(); assignment = {}; first = [None]
    use_fc = method != 'plain'; mrv = 'mrv' in method; deg = 'deg' in method; lcv = 'lcv' in method
    def select(domains):
        un = [v for v in vars_ if v not in assignment]
        if not mrv: return un[0]
        return min(un, key=lambda v: (len(domains[v]), -sum(1 for u in nb[v] if u not in assignment) if deg else 0, vars_.index(v)))
    def order_values(v, domains):
        vals = sorted(domains[v])
        if not lcv: return vals
        ruled = lambda x: sum(1 for u in nb[v] if u not in assignment for y in domains[u] if not ok(v, x, u, y))
        return sorted(vals, key=lambda x: (ruled(x), x))
    def rec(domains):
        if len(assignment) == len(vars_):
            st.solutions += 1
            if first[0] is None: first[0] = dict(assignment)
            return not all_solutions
        if limit and st.assign > limit: raise TimeoutError
        v = select(domains)
        for x in order_values(v, domains):
            st.assign += 1
            if not use_fc and not all(ok(v, x, u, assignment[u]) for u in nb[v] if u in assignment): continue
            assignment[v] = x
            if use_fc:
                new = {k: set(s) for k, s in domains.items()}; new[v] = {x}; wipe = False
                for u in nb[v]:
                    if u not in assignment:
                        new[u] = {y for y in new[u] if ok(v, x, u, y)}
                        if not new[u]: wipe = True; break
                if not wipe and rec(new): return True
            elif rec(domains): return True
            del assignment[v]
        st.back += 1
        return False
    rec({v: set(dom[v]) for v in vars_}); return first[0], st
map_ok = lambda a, x, b, y: x != y
```

```
def queens(n):
    V = list(range(n)); dom = {c: list(range(n)) for c in V}; nb = {c: [d for d in V if d != c] for c in V}
    return V, dom, nb, (lambda a, x, b, y: x != y and abs(x - y) != abs(a - b))
# ---------------- tree-structured CSP (AIMA 6.5) ----------------
TREE_PARENT = {'Maths': None, 'Physics': 'Maths', 'Programming': 'Maths', 'Mechanics': 'Physics', 'Optics': 'Physics',
               'Algorithms': 'Programming', 'Databases': 'Programming'}
TREE_DOM = {'Maths': {1,2,3,4}, 'Physics': {1,2,3,4}, 'Programming': {1,2,3,4}, 'Mechanics': {1,2,3,4}, 'Optics': {1,2,3,4},
            'Algorithms': {4}, 'Databases': {2,3}}          # unary limits: Algorithms only in slot 4, Databases in slot 2 or 3
def tree_ok(p, x, c, y): return x < y                      # a prerequisite exam must be earlier than the dependent exam
def solve_tree():
    """order: parent before child; (1) directional arc consistency leaf -> root; (2) assign root -> leaf. No backtracking."""
    order = ['Maths','Physics','Programming','Mechanics','Optics','Algorithms','Databases']
    D = {k: set(v) for k, v in TREE_DOM.items()}; log = []
    for X in reversed(order[1:]):
        P = TREE_PARENT[X]; before = sorted(D[P])
        D[P] = {x for x in D[P] if any(tree_ok(P, x, X, y) for y in D[X])}      # make arc (P, X) consistent
        log.append((P, X, before, sorted(D[P])))
    sol = {}; back = 0
    for X in order:
        P = TREE_PARENT[X]
        cand = sorted(D[X]) if P is None else [y for y in sorted(D[X]) if tree_ok(P, sol[P], X, y)]
        if not cand: back += 1; return None, log, back
        sol[X] = cand[0]
    return sol, log, back
def naive_bt_tree(order):
    """plain backtracking on the same tree CSP with a given variable order (counts failed subtrees)."""
    sol = {}; st = {'assign': 0, 'back': 0}
    def rec(i):
        if i == len(order): return True
        X = order[i]
        for x in sorted(TREE_DOM[X]):
            st['assign'] += 1
            ok = True
            for Y, y in sol.items():
                if TREE_PARENT[X] == Y and not tree_ok(Y, y, X, x): ok = False
                if TREE_PARENT[Y] == X and not tree_ok(X, x, Y, y): ok = False
            if not ok: continue
            sol[X] = x
            if rec(i + 1): return True
            del sol[X]
        st['back'] += 1; return False
    rec(0); return dict(sol), st
```

queens.py

```
"""Tasks 6 and 7: n-queens -- exhaustive enumeration, hill climbing variants, simulated annealing, min-conflicts."""
import random, math, itertools, time
import numpy as np
# ---------- state: r[c] = row of the queen in column c ; h = number of attacking pairs ----------
def counts(r, n):
    row = np.zeros(n, int); d1 = np.zeros(2*n, int); d2 = np.zeros(2*n, int)
    for c, x in enumerate(r): row[x] += 1; d1[x - c + n] += 1; d2[x + c] += 1
    return row, d1, d2
def heuristic(r):
    n = len(r); row, d1, d2 = counts(r, n)
    return int(sum(k*(k-1)//2 for k in row) + sum(k*(k-1)//2 for k in d1) + sum(k*(k-1)//2 for k in d2))
def neighbour_conf(r, n, row, d1, d2):
    """conf[c, x] = number of OTHER queens attacking square (c, x) -> delta h of moving queen c to row x is conf[c,x]-conf[c,r[c]]"""
    C = np.arange(n)[:, None]; X = np.arange(n)[None, :]
    conf = row[X] + d1[X - C + n] + d2[X + C]
    cur = np.array(r)
    conf[np.arange(n), cur] -= 3                        # remove the queen's own contribution
    return conf
def hill_climb(n, rng, sideways=0, max_steps=100000):
    """steepest-ascent hill climbing; sideways = max consecutive sideways moves. Returns (success, steps, evaluations)."""
    r = [rng.randrange(n) for _ in range(n)]; row, d1, d2 = counts(r, n); h = heuristic(r); steps = 0; side = 0; evals = 0
    while h > 0 and steps < max_steps:
        conf = neighbour_conf(r, n, row, d1, d2); evals += n * (n - 1)
        cur = np.array(r); base = conf[np.arange(n), cur][:, None]
        delta = conf - base; delta[np.arange(n), cur] = 10**9               # exclude the 'stay' entries
        best = delta.min()
        if best > 0 or (best == 0 and side >= sideways): break
        cs, xs = np.where(delta == best); k = rng.randrange(len(cs)); c, x = int(cs[k]), int(xs[k])
        old = r[c]; row[old] -= 1; d1[old - c + n] -= 1; d2[old + c] -= 1
        r[c] = x; row[x] += 1; d1[x - c + n] += 1; d2[x + c] += 1; h += int(best); steps += 1
        side = side + 1 if best == 0 else 0
    return h == 0, steps, evals, r
def anneal(n, rng, T0=2.0, cool=0.995, max_iter=200000):
    r = [rng.randrange(n) for _ in range(n)]; row, d1, d2 = counts(r, n); h = heuristic(r); T = T0; it = 0
    while h > 0 and it < max_iter:
        it += 1; T = max(T * cool, 1e-3)
        c = rng.randrange(n); x = rng.randrange(n)
        if x == r[c]: continue
        old = r[c]
        c_new = row[x] + d1[x - c + n] + d2[x + c]; c_old = row[old] + d1[old - c + n] + d2[old + c] - 3
        delta = c_new - c_old
        if delta <= 0 or rng.random() < math.exp(-delta / T):
            row[old] -= 1; d1[old - c + n] -= 1; d2[old + c] -= 1
            r[c] = x; row[x] += 1; d1[x - c + n] += 1; d2[x + c] += 1; h += delta
    return h == 0, it, r
def valid_board(r):
    n = len(r); return len(set(r)) == n and len({x - c for c, x in enumerate(r)}) == n and len({x + c for c, x in enumerate(r)}) == n
# ---------- exhaustive enumeration ----------
def all_solutions_backtracking(n):
    """returns (solutions, nodes): nodes = number of partial placements examined (bitmask backtracking, one queen per column)."""
    sols = []; nodes = [0]; full = (1 << n) - 1
    def rec(c, rows, d1, d2, cur):
        nodes[0] += 1
        if c == n: sols.append(tuple(cur)); return
        free = full & ~(rows | d1 | d2)
        while free:
            b = free & -free; free ^= b
            cur.append(b.bit_length() - 1)
            rec(c + 1, rows | b, ((d1 | b) << 1) & full, (d2 | b) >> 1, cur); cur.pop()
    rec(0, 0, 0, 0, []); return sols, nodes[0]
def symmetries(s):
    n = len(s); out = []
    pts = [(c, s[c]) for c in range(n)]
```

```
    for k in range(4):
        pts = [(y, n - 1 - x) for x, y in pts]                      # rotate 90 degrees
        for refl in (False, True):
            q = [(x, n - 1 - y) for x, y in pts] if refl else pts
            t = [0] * n
            for x, y in q: t[x] = y
            out.append(tuple(t))
    return out
# ---------- Task 7: original solver (plain backtracking, column by column) and the min-conflicts redesign ----------
def plain_bt(n):
    """colleague's solver: columns left to right, rows in ascending order, plain backtracking. Counts assignments, checks, backtracks."""
    r = [0] * n; st = {'assign': 0, 'checks': 0, 'back': 0, 'depth_nodes': [0] * n}
    def safe(c, x):
        for j in range(c):
            st['checks'] += 1
            if r[j] == x or abs(r[j] - x) == c - j: return False
        return True
    def rec(c):
        if c == n: return True
        for x in range(n):
            st['assign'] += 1; st['depth_nodes'][c] += 1
            if safe(c, x):
                r[c] = x
                if rec(c + 1): return True
        st['back'] += 1; return False
    ok = rec(0); return (r[:] if ok else None), st
def min_conflicts(n, rng, max_steps=100000):
    """one queen per column; repeatedly move a random conflicted queen to its minimum-conflict row (ties random).
       incremental counters make each conflict evaluation O(1). Returns (board or None, steps, conflict_evaluations)."""
    # greedy-ish initial assignment keeps the number of steps small (as in AIMA): place each queen at its min-conflict row
    row = np.zeros(n, int); d1 = np.zeros(2*n, int); d2 = np.zeros(2*n, int); r = [0]*n; evals = 0
    for c in range(n):
        confs = row + d1[np.arange(n) - c + n] + d2[np.arange(n) + c]; evals += n
        m = confs.min(); x = int(rng.choice(list(np.flatnonzero(confs == m)))); r[c] = x
        row[x] += 1; d1[x - c + n] += 1; d2[x + c] += 1
    steps = 0
    def conflicted():
        return [c for c in range(n) if row[r[c]] + d1[r[c] - c + n] + d2[r[c] + c] - 3 > 0]
    bad = conflicted()
    while bad and steps < max_steps:
        c = rng.choice(bad); old = r[c]
        row[old] -= 1; d1[old - c + n] -= 1; d2[old + c] -= 1
        confs = row + d1[np.arange(n) - c + n] + d2[np.arange(n) + c]; evals += n
        m = confs.min(); x = int(rng.choice(list(np.flatnonzero(confs == m)))); r[c] = x
        row[x] += 1; d1[x - c + n] += 1; d2[x + c] += 1; steps += 1
        bad = conflicted()
    return (r if not bad else None), steps, evals

def mc_restart(n, rng, cap=None):
    """min-conflicts with random restart: a run is abandoned after `cap` steps (plateau / cycle) and a new run is started."""
    cap = cap or max(100, 5 * n); total_steps = 0; total_evals = 0; restarts = 0
    while True:
        b, steps, ev = min_conflicts(n, rng, max_steps=cap); total_steps += steps; total_evals += ev
        if b is not None: return b, total_steps, total_evals, restarts
        restarts += 1
```